

# 数据结构第二次实验

郭威威2414308

## 根据next数组推导模式字符串

### 实验报告

#### 一、实验目的

本次实验的目的是实现一个程序，根据给定的首尾字符和 `next` 数组来构建一个模式字符串，并验证输入的 `next` 数组是否合法。

#### 二、实验环境

- 操作系统：Windows 11
- 编译器：g++
- 开发环境：VS Code

#### 三、实验步骤

1. 输入处理：从标准输入读取首尾字符和 `next` 数组的长度 `n`，然后读取 `next` 数组的每个元素。
2. 验证 `next` 数组：调用 `isValidNextArray` 函数验证 `next` 数组是否合法。
3. 构建模式字符串：如果 `next` 数组合法，则调用 `constructPattern` 函数根据首尾字符和 `next` 数组构建模式字符串。
4. 输出结果：输出构建好的模式字符串，如果 `next` 数组不合法或无法构建模式字符串，则输出 `"ERROR"`。

#### 四、实验结果

```
D:\Desktop\code\c\根据next3  X + v
1 0 10
-1 0 1 2 3 4 0 1 2 3
1111101110

-----
Process exited after 10.11 seconds with return value 0
请按任意键继续 . . . |
```

```
D:\Desktop\code\c\根据next3  X + v
1 1 6
-1 0 0 0 0 0
100001

-----
Process exited after 8.142 seconds with return value 0
请按任意键继续 . . . |
```

```
D:\Desktop\code\c\根据next  x + v
1 1 6
-1 0 2 0 0 0
ERROR

-----
Process exited after 2.706 seconds with return value 0
请按任意键继续 . . .
```

## 五、实验结论

通过本次实验，我们成功实现了根据首尾字符和 `next` 数组构建模式字符串的功能，并验证了输入的 `next` 数组的合法性。程序能够正确处理合法和不合法的输入，并输出相应的结果。

# 字符串应用-实现KMP匹配算法

## 实验报告

### 一、实验目的

本次实验的目的是实现 KMP（Knuth-Morris-Pratt）字符串匹配算法，用于在一个主串中查找指定模式串的位置。通过该实验，深入理解 KMP 算法的原理和实现，掌握 `next` 数组的计算方法，以及如何利用 `next` 数组进行高效的字符串匹配。

### 二、实验环境

- 操作系统：Windows 11
- 编译器：g++
- 开发环境：VS Code

### 三、实验原理

KMP 算法是一种高效的字符串匹配算法，其核心思想是利用已经匹配过的信息，避免在匹配过程中进行不必要的回溯。具体来说，KMP 算法通过计算模式串的 `next` 数组，记录模式串中每个位置之前的子串的最长公共前后缀长度。在匹配过程中，当发生不匹配时，利用 `next` 数组将模式串向右移动一定的距离，从而减少比较次数，提高匹配效率。

## 四、实验步骤

1. 输入处理：从标准输入读取主串 `S` 和模式串 `P`，主串以空格为分隔符。
2. 计算 `next` 数组：调用 `computeNext` 函数计算模式串 `P` 的 `next` 数组。
3. 字符串匹配：调用 `kmpSearch` 函数，使用 KMP 算法在主串 `S` 中查找模式串 `P` 的位置。
4. 输出结果：如果找到匹配，输出匹配位置的序号（从 0 开始）；如果未找到匹配，输出 "no"。

## 五、实验结果

```
D:\Desktop\code\c\字符串搜索 × + ▾
ababcabcacbab abcac
5
-----
Process exited after 4.261 seconds with return value 0
请按任意键继续 . . .
```

```
D:\Desktop\code\c\字符串搜索 × + ▾
ABCDABCDABDE DBAEA
no
-----
Process exited after 5.639 seconds with return value 0
请按任意键继续 . . .
```

## 六、实验结论

通过本次实验，成功实现了 KMP 字符串匹配算法。实验结果表明，KMP 算法能够高效地在主串中查找模式串的位置，避免了传统暴力匹配算法中不必要的回溯，提高了匹配效率。同时，通过计算 `next` 数组，充分利用了模式串的内部信息，使得匹配过程更加智能和高效。

## 七、实验总结

在实验过程中，对 KMP 算法的原理和实现有了更深入的理解。`next` 数组的计算是 KMP 算法的关键，它记录了模式串的重要信息，能够指导模式串在匹配过程中的移动。在实现过程中，需要注意边界条件的处理和指针的移动，确保算法的正确性。同时，通过测试不同的输入用例，验证了算法的正确性和有效性。

此外，KMP 算法的时间复杂度为  $O(n + m)$ ，其中  $n$  是主串的长度， $m$  是模式串的长度，相比传统的暴力匹配算法  $O(nm)$  有了显著的性能提升。在实际应用中，KMP 算法可以用于文本搜索、字符串匹配等场景，具有重要的实用价值。